CCEE Java OOPS Module – Comprehensive Cheat Sheet

1. Java Fundamentals (Sessions 1 & 2)

Architecture

- **WORA:** "Write Once, Run Anywhere" – Code is compiled to Bytecode, which runs on any JVM.
- **JDK (Java Development Kit):** For **Developers**. Contains JRE + Tools (`javac` , `jdb`).
- **JRE (Java Runtime Environment):** For **Users**. Contains JVM + Libraries.
- **JVM (Java Virtual Machine):** The engine that executes Bytecode. **Platform Dependent**.

Data Types & Variables

- **Primitive Types (8):** `byte` , `short` , `int` , `long` , `float` , `double` , `boolean` , `char` . Stored in **Stack**.
- **Reference Variables:** Store the **address** of the object. The object itself lives in the **Heap**.
- **Variable Scope:**
  1. **Local:** Inside a method. **NO Default Value**. Must initialize before use (or compile error).
  2. **Instance:** Inside a class. Has **Default Value** (0, 0.0, null, false).
  3. **Static:** Shared across all instances. Loaded once.

2. Object Lifecycle & wrappers (Session 3)

Lifecycle

1. **Creation:** `new ClassName()` allocates memory on the Heap.
2. **Access:** via `.` operator.
3. **Garbage Collection (GC):** Automatic memory management.
   - **Eligibility:** When an object is **unreachable** (ref is `null` or reassigned).
   - `finalize()` : Called before destruction (Deprecated).

Wrapper Classes

- **Purpose:** Treat primitives as Objects (needed for Collections).
- **Autoboxing:** Automatic `int` → `Integer` .

- **Unboxing:** Automatic `Integer` → `int`.
- **Null Trap:** Unboxing a `null` Wrapper throws `NullPointerException`.

Operators & Control

- **Increment:** `x++` (Use then Increment) vs `++x` (Increment then Use).
- **Switch:** Supports `int`, `char`, `String`, `Enum`. **Does NOT support** `float`, `double`, `boolean`.
- **Equality:** `==` compares references (address) for objects, values for primitives. `.equals()` compares content (if overridden).

3. Arrays & Strings (Session 4)

Arrays

- **Fixed Size:** Cannot change length after creation.
- **Declaration:** `int[] arr = new int[5];` (Indices 0 to 4).
- **Accessing Out of Bounds:** Throws `ArrayIndexOutOfBoundsException`.

Strings (The Special Class)

- **Immutability:** String objects cannot be changed.
- **String Constant Pool (SCP):** Special memory area for literals. `String s = "Hello"` uses SCP. `new String("Hello")` forces a new Heap object.
- **Mutable Alternatives:**
  - **StringBuilder:** Fast, Not Thread-Safe (Use this usually).
  - **StringBuffer:** Slower, Thread-Safe (Synchronized).

4. OOPS Core: Encapsulation & Methods (Session 4)

Access Modifiers (Visibility)

1. **private:** Class only.
2. **default (no keyword):** Package only.
3. **protected:** Package + Subclasses (even in other packages).
4. **public:** Everyone.

Constructors

- Same name as class, no return type.

- **Default Constructor:** Provided by compiler **only if** no other constructor is defined.
- `this()` : Calls another constructor in the same class (Must be 1st line).
- `super()` : Calls parent constructor (Must be 1st line).
- **Constraint:** You cannot use `this()` and `super()` in the same constructor.

5. Inheritance & Polymorphism (Session 5)

Inheritance

- **Keyword:** `extends` .
- **Multiple Inheritance:** NOT supported for classes (to avoid Diamond Problem).
- **Object Casting:**
  - **Upcasting:** Child → Parent (Automatic).
  - **Downcasting:** Parent → Child (Manual, risk of `ClassCastException`).

Polymorphism

- **Compile-Time (Overloading):** Same method name, **different parameter list**. Return type doesn't matter.
- **Run-Time (Overriding):** Same method name, same parameters, parent-child relationship.
  - **Virtual Methods:** All non-static methods in Java are virtual by default (overridden based on actual object type).
  - **Static/Final/Private:** Cannot be overridden.

Abstract Class vs Interface

- **Abstract Class:** Can have state (fields) and constructors. Use for "is-a" relationship.
- **Interface:** Abstract methods (mostly). Java 8 allows `default` and `static` methods. Use for "capabilities" (e.g., `Runnable`).

6. Exception Handling (Session 6)

Hierarchy

- **Throwable** → **Error** (Don't catch, e.g., OutOfMemory) & **Exception**.
- **Checked Exception:** Checked at compile time (e.g., `IOException`). Must be handled.
- **Unchecked (Runtime) Exception:** e.g., `NullPointerException` , `ArithmeticException` .

Keywords

- `try-catch-finally` : Handle errors. `finally` always runs.
- `throw` : Explicitly throw an exception.
- `throws` : Declare exception in method signature.
- **Custom Exceptions:** Extend `Exception` (Checked) or `RuntimeException` (Unchecked).

7. Advanced Syntax (Session 7)

Enums

- Fixed set of constants.
- Can have fields, methods, and constructors (always private).
- Methods: `values()` , `ordinal()` .

Annotations

- Metadata for compiler/runtime.
- Common: `@Override` , `@Deprecated` , `@SuppressWarnings` .

8. Packages (Session 8)

`java.lang`

- Imported **automatically**.
- Includes: `Object` , `String` , `Math` , `System` , `Thread` , `Integer` .
- `Object` **methods:** `toString()` , `equals()` , `hashCode()` , `clone()` , `getClass()` .

`java.util`

- Must be imported.
- Includes: Collections Framework, `Scanner` , `Date` , `Random` .

9. Generics & Collections (Sessions 9 & 10)

Generics

- Enforces **Type Safety** at compile time.
- **Non-Covariant:** `List<Object>` is NOT a parent of `List<Integer>` .
- **Diamond Operator:** `List<String> l = new ArrayList<>();`

The Framework

A. List (Ordered, Duplicates OK)

- **ArrayList:** Fast random access ($O(1)$). Slow insertions. Backed by Array.
- **LinkedList:** Fast insertions/deletions. Slow access. Backed by Nodes.

B. Set (Unordered, Unique only)

- **HashSet:** Uses Hashing. Unordered. Fastest.
- **LinkedHashSet:** Maintains **Insertion Order**.
- **TreeSet:** Maintains **Sorted Order** (Natural/Comparator).

C. Map (Key-Value, Keys Unique)

- **HashMap:** Fast. Allows 1 `null` key. Not synchronized.
- **Hashtable:** Legacy. Thread-safe (slow). No `null` s allowed.
- **TreeMap:** Sorted by Keys.

Exam Gotchas

1. **Compare Objects:** Always use `.equals()` , not `==` .
2. **Looping:** Removing items from a List while iterating with a `for-each` loop throws `ConcurrentModificationException` . Use `Iterator.remove()` .
3. **Local Variables:** Just declaring `int x;` inside main and trying to print it is a Compile Error.